

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2-DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	JEEVAN RAAJA S	Reg.No.:	192421005
Department:	BTECH IT	Date:	31.7.2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I Jeevan Raaja S (192421005) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Jeevan raaja s

MARKS ALLOCATION

	Identification of Errors / Bugs(20%)	Debugging Approach & Analysis(20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied(20%)	Testing and Documentation(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable	Limited optimization with minimal	No optimization applied or inefficient implementation

1. Debugging Signed Integer Addition (2's Complement Overflow)

Problem

Add +120 and +50 using 8-bit 2's complement.

Step 1: Convert to Binary

+120 = 01111000

+50 = 00110010

Step 2: Binary Addition

```
  01111000
+ 00110010
-----
  10101010
```

Step 3: Decimal Result

10101010 (2's complement)

Invert bits:

```
  10101010
  01010101
+          1
-----
  01010110 = 86
```

Result = -86

Expected result:

120 + 50 = 170

Since 170 cannot be represented in an 8-bit signed integer (range -128 to +127), **overflow has occurred**.

Debugging

- Two positive numbers produced a negative result.
- Carry into the sign bit $\neq$ Carry out of the sign bit.
- Overflow flag should be set.

Solution

Detect overflow using:

Overflow = Carry into MSB $\oplus$ Carry out of MSB

or

If operands have the same sign
AND result has a different sign
→ Overflow

Final Answer

- **Overflow:** Yes
- **Problem:** Result exceeds +127.
- **Correction:** Detect overflow using the overflow flag and report an arithmetic overflow instead of accepting the incorrect result.

2. Debugging Ripple Carry Adder Delay

Problem

Ripple Carry Adder (RCA) is slow because the carry must pass through every bit before the final sum is produced.

Example (4-bit)

A = 1011
B = 0110

Carry propagation:

```
Bit0 → Carry
      ↓
Bit1 → Carry
      ↓
Bit2 → Carry
      ↓
Bit3 → Final Carry
```

Each stage waits for the previous carry.

Debugging

- Delay increases with the number of bits.
- Worst-case delay occurs when every stage generates a carry.

Optimized Solution

Use a **Carry Look Ahead Adder (CLA)**.

CLA computes carries simultaneously using:

Generate (G) = A × B

Propagate (P) = A ⊕ B

Carry equations:

C1 = G0 + P0C0

C2 = G1 + P1G0 + P1P0C0

C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0

Benefits

- Parallel carry generation
- Faster addition
- Reduced propagation delay
- Better performance in real-time systems

Final Answer

- **Problem:** Sequential carry propagation.
- **Solution:** Replace RCA with CLA.
- **Benefit:** Significantly reduces arithmetic delay.

- Perform arithmetic right shift.
- Preserve sign bit.

Debugging

Possible errors:

- Incorrect bit-pair checking.
- Logical shift used instead of arithmetic shift.
- Incorrect sign extension.
- Incorrect initialization of Q₋₁.
- Wrong subtraction using 2's complement.

Corrections

- Use arithmetic right shift.
- Preserve MSB during shifting.
- Initialize Q₋₁ = 0.
- Use proper 2's complement subtraction.
- Verify bit-pair decisions.

Final Answer

- **Problem:** Incorrect handling of signed numbers.
- **Solution:** Correct bit-pair checking, arithmetic shifts, and sign extension.
- **Result:** Accurate signed multiplication.

3. Debugging Booth's

Multiplication Algorithm

Problem

Booth's algorithm gives incorrect results for negative operands.

Example

Multiply

Multiplicand = -5

Multiplier = 3

Booth's Rules

Q _n Q _{n-1}	Operation
00	No operation
01	Add M
10	Subtract M
11	No operation

After every operation:

4. Debugging Restoring

Division Algorithm

Problem

Restoring division repeatedly restores the previous value after unsuccessful subtraction, increasing execution time.

Workflow

Shift left

↓

Subtract divisor

↓

Negative?

Yes

↓

Restore

↓

Next iteration

Debugging

Time is wasted because:

- Every negative subtraction requires restoration.
- Extra addition is performed.
- More clock cycles are needed.

Optimized Solution

Use the **Non-Restoring Division Algorithm**.

Workflow:

Shift

↓

Subtract

↓

Negative?

Next cycle performs addition

(No restoration)

Advantages

- Eliminates restoration step.
- Fewer arithmetic operations.
- Faster execution.
- Lower power consumption.
- Better for embedded systems.

Final Answer

- **Problem:** Repeated restoration operations.
- **Solution:** Replace with Non-Restoring Division.
- **Benefit:** Faster and more efficient division.

5. Debugging IEEE 754 Floating-Point Addition

Problem

Adding a very large number and a very small number causes inaccurate results.

Example:

$$1.0 \times 10^{20}$$

+

$$1.0$$

Step 1: Exponent Alignment

$$1.0 \times 10^{20}$$

+

$$0.00000000000000000001 \times 10^{20}$$

The smaller number's mantissa is shifted right many places.

Step 2: Addition

The shifted mantissa becomes too small to fit within the 23-bit fraction.

It is rounded away.

Result:

$$\approx 1.0 \times 10^{20}$$

The value **1.0** is effectively lost.

Debugging

Errors arise due to:

- Exponent alignment
- Loss of precision
- Normalization
- Rounding

Optimized Solution

- Use **double precision (64-bit)** instead of single precision.
- Apply compensated summation algorithms (e.g., **Kahan Summation**) to reduce rounding errors.
- Sum numbers from smallest to largest when possible.

Benefits

- Higher precision
- Reduced rounding error
- Improved numerical stability
- Better accuracy for scientific applications

Final Answer

- **Problem:** Precision loss during exponent alignment and rounding.
- **Solution:** Use double precision and improved summation algorithms.

- **Benefit:** More accurate floating-point computations.